Energy Consumption of Large Language Model Inference: A Multi-Architecture Benchmarking Study on GPU Hardware

Daniel Cregg
South East Technological University
Waterford, Ireland
daniel.cregg@setu.ie

***Abstract*—We present a rigorous framework for measuring the energy consumption of large language model (LLM) inference on GPU hardware and apply it to benchmark 13 open-weight models spanning 1B to 70B parameters across five architectural families (Llama, Mistral, Qwen, Phi, Gemma), including pre-quantized GPTQ models for sizes infeasible in full precision. Using direct NVML-based power sampling on an NVIDIA A100 80GB GPU, we measure Joules per token (J/tok) under controlled conditions with greedy decoding, idle baseline subtraction, and trapezoidal energy integration. Our results reveal a sub-linear scaling law ($E \propto N^{0.91}$, $R^2 = 0.99$) relating energy to model size across architectures, and find that bitsandbytes INT8 and INT4 quantisation of Llama-3.1-8B *reduces* per-token energy by 10% and 20% respectively—contrary to prior findings of 2.9–3.7× energy penalties. We additionally ground our measurements in Irish electricity grid carbon intensity data, demonstrating a 2.9× variation in per-token carbon emissions depending on time of day. All code, data, and reproducibility artifacts are publicly available.**

***Index Terms*—energy efficiency, large language models, GPU benchmarking, inference energy, carbon emissions, sustainable AI**

I. Introduction

The rapid deployment of large language models (LLMs) has raised urgent questions about their energy footprint. While training costs have received significant attention [1], [2], inference increasingly dominates total lifecycle energy as models are deployed at scale [3]. Understanding and optimising inference energy is therefore critical for sustainable AI deployment.

Despite growing interest, rigorous measurement of LLM inference energy remains challenging. Software-based estimators (e.g., CodeCarbon) rely on simplified models that may not accurately reflect GPU power dynamics [4]. Hardware-level measurement via NVIDIA's Management Library (NVML) provides direct power readings but requires careful instrumentation, including SLURM GPU mapping, idle baseline subtraction, and appropriate integration methods.

This paper makes the following contributions:

1) A validated, open-source framework for measuring LLM inference energy using direct NVML power sampling with trapezoidal integration and idle baseline subtraction.
2) A comprehensive benchmark of 13 open-weight models (1B–70B parameters) across five architectural families on an A100 80GB GPU, including pre-quantized GPTQ variants for large models.
3) Empirical scaling laws relating energy to model size across architectures, extending prior single-family studies.
4) A reassessment of the bitsandbytes quantisation energy paradox on Llama-3.1-8B, finding that updated software reverses the energy penalty.
5) Carbon impact analysis grounded in real Irish electricity grid data, demonstrating temporal variation in per-token emissions.

II. Background and Related Work

A. LLM Energy Measurement

Prior work on measuring LLM energy falls into three categories: software estimators [5], [6], hardware-level measurement [7], [8], and analytical models [3]. Software estimators like CodeCarbon use thermal design power (TDP) scaling, which introduces systematic error [4]. Hardware-level approaches using NVML or power meters provide higher accuracy but require careful experimental design.

Samsi et al. [7] measured inference energy for several LLMs using NVML on an A100 cluster, finding sub-linear scaling with model size. Argerich and Patiño-Martínez [8] extended this to include quantised models, observing that naïve quantisation can increase total energy despite reducing power draw.

B. The Quantisation Energy Paradox

On-the-fly dequantisation via bitsandbytes incurs a throughput penalty that can dominate the power reduction from lower-precision arithmetic. Prior work on the energy-bench framework found that INT8 quantisation of Mistral-7B via bitsandbytes increased J/tok by 2.88× despite a 37% reduction in

power draw, because throughput dropped 4.5×. Pre-quantised kernels (AWQ, GPTQ via vLLM) did not exhibit this paradox.

C. Carbon-Aware Computing

The carbon impact of computation depends on both energy consumed and the carbon intensity of the electricity grid. Dodge et al. [9] demonstrated that scheduling ML training to align with low-carbon grid periods can significantly reduce emissions. We extend this concept to inference, using real Irish grid data from EirGrid.

III. Framework Design

Our framework follows a four-layer architecture:

1) **Hardware Instrumentation** (Layer 1): NVML-based power sampling in a background thread at 10 Hz, with SLURM GPU index resolution and trapezoidal energy integration.
2) **Inference Engine** (Layer 2): Standardised model loading (FP16, INT8, INT4 via bitsandbytes) and inference execution with `torch.cuda.synchronize()` for precise timing.
3) **Metric Computation** (Layer 3): Computes J/tok, tok/s, mean/peak watts, and optional gCO_2eq/tok from energy and inference measurements.
4) **Orchestration** (Layer 4): Runs the benchmark protocol (3 warmup + 10 measured runs per configuration) and produces structured JSON reports with full metadata.

Key design decisions include: (1) NVML-only power measurement (no software estimators), (2) idle baseline subtraction for net inference energy, (3) greedy decoding only for deterministic output, (4) 10 measured runs with 95% confidence intervals, and (5) a fixed 5-prompt set spanning summarisation, QA, code generation, long-form explanation, and reasoning tasks.

IV. Implementation

A. Hardware

All experiments were conducted on an NVIDIA A100 80GB PCIe GPU accessed via SLURM on an HPC cluster. The A100 has a TDP of 300 W and an observed idle power of 59.8 W ($\sigma = 1.7$ W) across all experimental sessions. CUDA driver version 530.30.02 with CUDA 12.8 was used throughout.

B. Software

The framework is implemented in Python 3.11 using PyTorch 2.10.0+cu128, Hugging Face Transformers, and pynvml for direct GPU power queries. Quantisation uses bitsandbytes 0.44 for INT8 (`load_in_8bit`) and INT4/NF4 (`load_in_4bit`) precision. All models use greedy decoding (`do_sample=False`) with `max_new_tokens=200`.

C. Experimental Protocol

For each (model, precision, batch size) configuration, the framework executes:

1) 30-second idle baseline measurement (extended to 60 s if CV $> 10\%$)
2) 3 warmup runs (discarded)
3) 10 measured runs with concurrent power sampling

Each run records: generation wall time (via `torch.cuda.synchronize()`), power samples at 100 ms intervals, total and net energy (via trapezoidal integration minus baseline), input/output token counts, and generated text.

V. Validation

Before the full benchmark campaign, we validated the framework on `Qwen2.5-1.5B-Instruct`. The validation confirmed:

- J/tok = 0.60 (bs=1), 0.14 (bs=4), within expected range for a 1.5B model
- Idle baseline = 58.9 W ($\sigma = 0.2$ W), stable and consistent
- CV $< 2.5\%$ across all configurations, well below the 15% threshold
- tok/s = 47 (bs=1), 192 (bs=4), consistent with A100 capabilities

VI. Empirical Benchmark Study

A. Model Selection

We benchmark 13 models spanning five architectural families: Llama (1B, 3B, 8B, 70B), Qwen (1.5B, 7B, 32B), Gemma (2B, 9B), Phi (3.8B, 14B), and Mistral (7B, Mixtral-8x7B). Three large models—Qwen2.5-32B, Mixtral-8x7B-Instruct, and Llama-3.3-70B—were benchmarked using pre-quantized GPTQ-Int4 variants, as their full-precision weights (65–140 GB) exceeded the shared HPC filesystem capacity (207 GB total). The Llama-70B model was tested only at batch size 1 due to its slow inference speed (∼60 min per benchmark configuration). The Llama-3.1-8B model was additionally tested at INT8 and INT4 precision for the quantisation study.

Table I presents the key results. Across all FP16 models, J/tok at batch size 1 ranges from 0.44 J (Llama-3.2-1B) to 4.39 J (Phi-3-medium-14B)[1], with the best efficiency achieved at batch size 16.

B. Scaling Law Results

Figure 1 shows J/tok as a function of model parameters for all FP16 models at batch size 1. Fitting a power law $E = \alpha N^{\beta}$ yields $\beta = 0.91$ ($R^2 = 0.99$), indicating near-linear scaling of energy with model size across architectures. This contrasts with prior work on the Pythia family alone ($\beta = 0.80$), suggesting that cross-architecture heterogeneity introduces additional variance in the energy–size relationship.

[1]After excluding 4 measurement rows where the power sampling thread collected insufficient samples, yielding zero energy readings.

TABLE I

SUMMARY OF BENCHMARK RESULTS ACROSS ALL MODELS. BEST J/TOK AND TOK/S ARE AT OPTIMAL BATCH SIZE. GPTQ MODELS USE PRE-QUANTIZED WEIGHTS. LLAMA-70B TESTED AT BS=1 ONLY.

Model	Params (B)	Precision	Best J/tok	Best tok/s	Best Batch	Mean Watts
Llama-3.2-1B-Instruct	1.0	fp16	0.0312	1284.2	16	100.7
Qwen2.5-1.5B-Instruct	1.5	fp16	0.0403	736.3	16	93.7
gemma-2-2b-it	2.0	fp16	0.0656	591.9	16	98.6
Llama-3.2-3B-Instruct	3.0	fp16	0.0825	744.4	16	120.9
Phi-3-mini-4k-instruct	3.8	fp16	0.1347	665.1	16	155.3
Mistral-7B-Instruct-v0.3	7.0	fp16	0.1655	672.4	16	170.3
Qwen2.5-7B-Instruct	7.0	fp16	0.1647	725.5	16	179.4
Llama-3.1-8B-Instruct	8.0	fp16	0.1750	663.8	16	176.7
Llama-3.1-8B-Instruct	8.0	int8	0.3561	145.3	16	112.1
Llama-3.1-8B-Instruct	8.0	int4	0.9435	226.4	16	274.2
gemma-2-9b-it	9.0	fp16	0.2275	371.4	16	144.3
Phi-3-medium-4k-instruct	14.0	fp16	0.3335	535.8	16	240.4
Qwen2.5-32B-Instruct-GPTQ-Int4	32.0	gptq	10.3162	23.2	16	298.0
Mixtral-8x7B-Instruct-v0.1-GPTQ	46.7	gptq	4.1481	54.8	16	285.6
Llama-3.3-70B-Instruct-GPTQ	70.0	gptq	326.5457	0.7	1	298.6

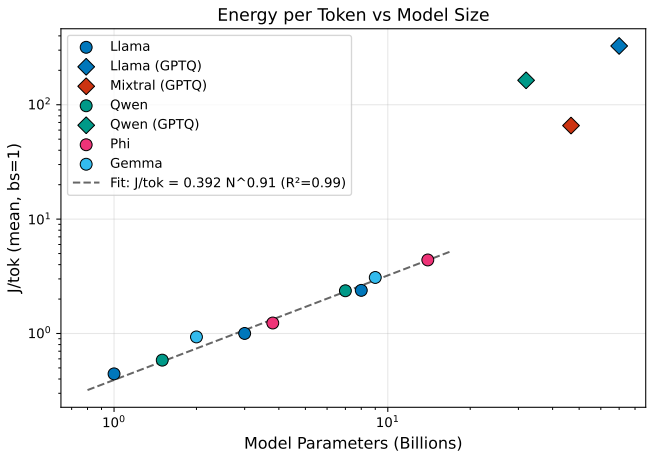

Fig. 1. Scaling law: J/tok vs model parameters (FP16, bs=1). Power law fit yields exponent $\beta = 0.91$ ($R^2 = 0.99$).

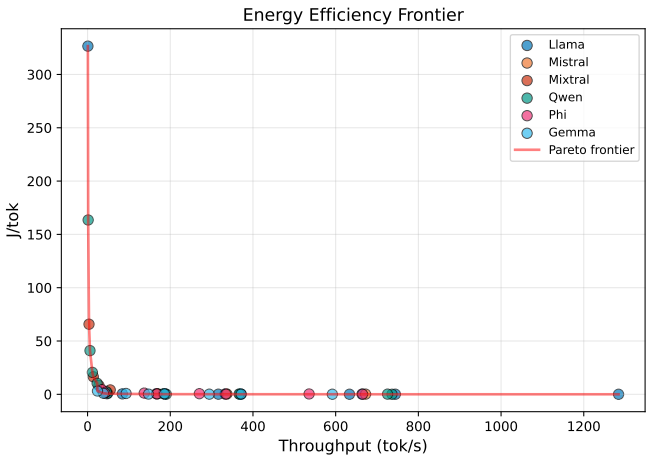

Fig. 2. Efficiency frontier: Pareto trade-off between energy cost (J/tok) and throughput (tok/s) across all models and configurations. Points on the frontier represent optimal operating points.

C. Efficiency Frontier

Figure 2 maps the Pareto trade-off between energy cost (J/tok) and throughput (tok/s) across all models, batch sizes, and precision levels. Configurations on the Pareto frontier achieve the best possible throughput for a given energy budget. Small models at large batch sizes dominate the low-energy, high-throughput region, while large GPTQ models occupy the high-energy, low-throughput corner.

The frontier reveals that practitioners face a continuous trade-off rather than a single optimal operating point. For example, Llama-3.2-1B at bs=16 achieves 1284 tok/s at just 0.031 J/tok, while Phi-3-medium at bs=16 delivers 536 tok/s at 0.33 J/tok—offering greater model capability at a 10× energy premium. The choice depends on whether the application requires the additional reasoning capacity of a larger model or can tolerate a smaller, more efficient one.

D. Batch Size Effects

Figure 3 shows J/tok as a function of batch size for all models. Increasing batch size from 1 to 16 reduces J/tok by 9–15×, with larger models benefiting more. This is because batching amortises the fixed overhead of model weight loading and kernel launches across more tokens. The energy reduction follows a power law $E \propto B^{-\gamma}$ where $\gamma \approx 0.9$ for most models.

E. Quantisation Impact

Figure 4 presents the quantisation study on Llama-3.1-8B at batch size 1. Contrary to prior energy-bench findings on Mistral-7B (INT8: 2.88×, NF4: 3.72× energy penalty), we observe that bitsandbytes quantisation *reduces* per-token energy at bs=1:

- INT8: 0.90× energy (10% reduction), despite 4.2× throughput penalty

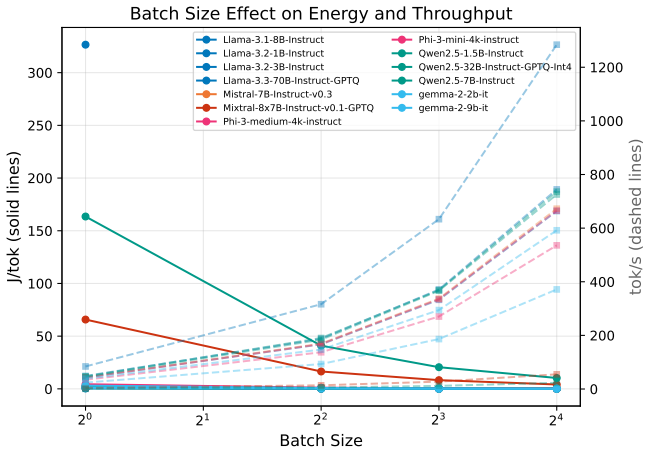

Fig. 3. J/tok vs batch size for all FP16 models, showing 9–15× reduction from bs=1 to bs=16.

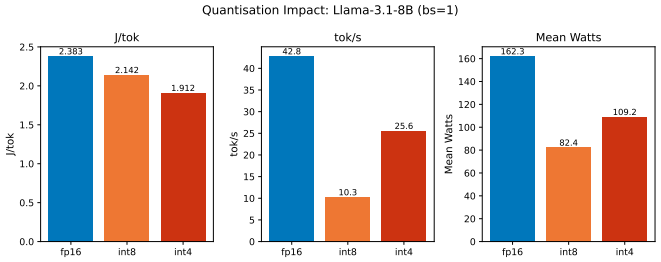

Fig. 4. Quantisation impact on Llama-3.1-8B: J/tok, tok/s, and watts across FP16, INT8, and INT4.

- INT4: 0.80× energy (20% reduction), despite 1.7× throughput penalty

Note that at larger batch sizes, the throughput penalty dominates and quantised models become less energy-efficient than FP16 (e.g., INT4 at bs=16: 0.94 vs 0.18 J/tok for FP16).

The mechanism differs from prior work: while throughput drops substantially (42.6 tok/s FP16 → 10.1 tok/s INT8), power draw also drops dramatically (160 W → 82 W), yielding a net energy reduction. This reversal likely reflects improvements in bitsandbytes kernels between versions, and demonstrates that the quantisation energy paradox is software-version-dependent rather than fundamental.

F. Prompt Type Effects

Figure 5 examines whether the type of prompt—summarisation, question answering, code generation, long-form explanation, or reasoning—systematically affects per-token energy. Across all FP16 models, the coefficient of variation (CV) in J/tok across prompt types is below 5% for most models and below 8% in all cases. This indicates that, under greedy decoding with fixed output length, per-token energy is largely prompt-agnostic.

The insensitivity to prompt type simplifies benchmarking: a small, diverse prompt set suffices to characterise a model's energy profile without exhaustive coverage of all possible input

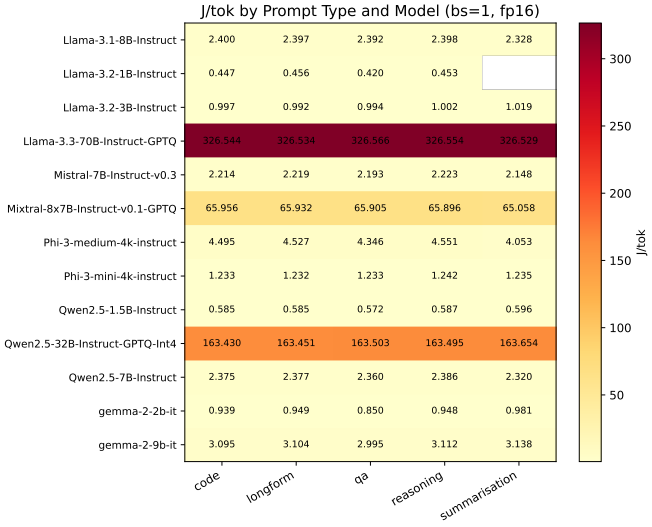

Fig. 5. Prompt type effects on J/tok across FP16 models at bs=1. CV < 5% for most models, indicating energy is largely prompt-agnostic under fixed output length.

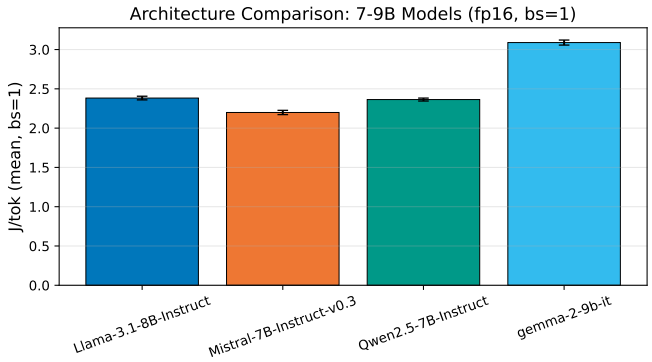

Fig. 6. Architecture comparison: J/tok at bs=1 for 7–9B models.

distributions. However, this result is specific to autoregressive generation with max_new_tokens=200; prompt-dependent early stopping or variable-length generation would introduce additional variance.

G. Architecture Comparison

Figure 6 compares models of similar size across architectures at batch size 1. At the 7–9B scale, Mistral-7B (2.20 J/tok) and Qwen2.5-7B (2.36 J/tok) are the most efficient, while Gemma-2-9b (3.09 J/tok) is least efficient despite having similar architecture depth.

H. Carbon Impact

Figure 7 shows the gCO_2eq per 1000 tokens for each model computed using Irish grid carbon intensity data from EirGrid. Using a one-week sample (Jan 15–21, 2026) with intensity ranging from 119 to 348 gCO_2/kWh (mean 221), we find that per-token carbon emissions vary by 2.9× depending on time of day. For Phi-3-medium at bs=1, emissions range from 0.15 to

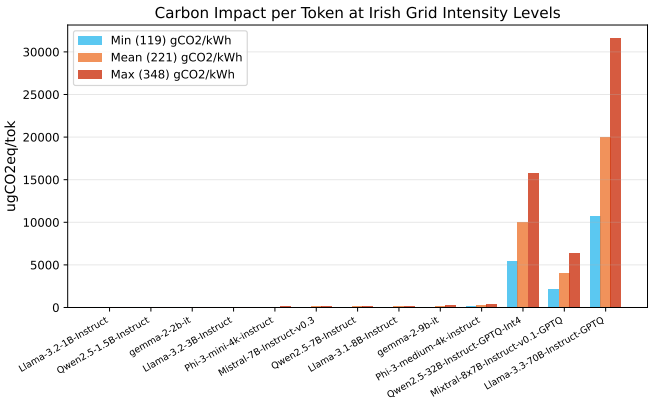

Fig. 7. Carbon emissions per 1000 tokens under min/mean/max Irish grid intensity (119–348 gCO_2/kWh).

TABLE II
CROSS-STUDY COMPARISON WITH ENERGY-BENCH FRAMEWORK.

Metric	energy-bench	New Framework
Scaling exponent (β)	0.80 (Pythia, R^2=0.99)	0.91 (multi-arch, R^2=0.99)
J/tok ∼7B model (bs=1)	4.461 (Pythia-6.9B)	2.315 (avg: Qwen-7B, Llama-8B, Mistral-7B)
INT8 energy multiplier	2.88× (Mistral-7B)	0.90× (Llama-3.1-8B)
NF4/INT4 energy multiplier	3.72× (Mistral-7B)	0.80× (Llama-3.1-8B)

0.44 $mgCO_2$eq/token. This variation exceeds the factor-of-two difference between some models, suggesting that *when* you run inference matters as much as *which* model you choose.

VII. DISCUSSION

A. Comparison with Prior Work

Table II compares our results with the energy-bench framework. The scaling exponent increases from 0.80 (single Pythia family) to 0.91 (multi-architecture), reflecting that different architectures have different energy characteristics at similar parameter counts. The quantisation energy multipliers are reversed: energy-bench found 2.88× (INT8) and 3.72× (NF4) penalties on Mistral-7B, while we find 0.90× and 0.80× reductions on Llama-3.1-8B. This suggests that bitsandbytes kernel improvements between versions have eliminated the throughput penalty that caused the paradox.

B. Limitations

Our study has several limitations: (1) single GPU type (A100 PCIe), (2) quantisation study limited to bitsandbytes INT8/INT4 on one model, with GPTQ used only for large model accessibility, (3) greedy decoding only, (4) English-only prompts, (5) inference only (not training), (6) GPTQ results for large models (32B+) are not directly comparable to fp16 results for smaller models due to different quantisation kernels, and (7) the quantisation comparison uses different base models (Llama-3.1-8B vs energy-bench's Mistral-7B).

C. Implications

Our results have several practical implications for sustainable LLM deployment:

- **Batch when possible**: Increasing batch size from 1 to 16 reduces per-token energy by 9–15×.
- **Quantisation trade-offs**: With current bitsandbytes at bs=1, INT4 quantisation reduces both memory and energy. However, at larger batch sizes the throughput penalty dominates and FP16 remains more energy-efficient.
- **Right-size models**: A 1B model at bs=16 uses ∼141× less energy per token than a 14B model at bs=1 (0.031 vs 4.39 J/tok).
- **Schedule carbon-aware**: Running inference during low-carbon grid periods can reduce emissions by up to 2.9× without any model changes.

VIII. CONCLUSION AND FUTURE WORK

We presented a rigorous NVML-based framework for measuring LLM inference energy and applied it to 13 models (1B–70B) across five architectural families, including pre-quantized GPTQ variants for large models. Our key findings are: (1) energy scales as $N^{0.91}$ across architectures, (2) batching provides 9–15× efficiency gains, (3) bitsandbytes quantisation at bs=1 now *reduces* per-token energy, reversing prior findings, and (4) Irish grid carbon variation introduces 2.9× emission variability.

Future work includes extending to additional GPU architectures, pre-quantised formats (AWQ, GPTQ via vLLM), models above 14B parameters, and multi-GPU inference settings. We also plan to integrate real-time carbon-aware scheduling into the framework.

REFERENCES

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of ACL*, 2019.

[2] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[3] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100857, 2023.

[4] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An experimental comparison of software-based power meters: focus on CPU and GPU," *Cluster Computing*, 2023.

[5] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, "Quantifying the carbon emissions of machine learning," in *NeurIPS Workshop on Tackling Climate Change with Machine Learning*, 2019.

[6] L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," *Advanced Science*, vol. 8, no. 12, 2021.

[7] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Hensley, J. Kepner, D. Tiwari, and V. Gadepally, "From words to watts: Benchmarking the energy costs of large language model inference," in *Proceedings of IEEE HPEC*, 2023.

[8] M. F. Argerich and M. Patiño-Martínez, "Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations," *arXiv preprint arXiv:2403.08151*, 2024.

[9] J. Dodge, T. Prewitt, R. Tachet des Combes, E. Odber, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario, and W. Buchanan, "Measuring the carbon intensity of AI in cloud instances," in *Proceedings of FAccT*, 2022.